Leo Alves

engineer & (accidental) indie researcher

Azul Labs

AI and Salesforce consulting.

leo@azl.au · azl.au

Fine tuning

01

Everyone wants frontier level capability.

But...

- You might not need frontier level capability at every task.
- Behaviour/style/speech is defined by a different organisation.
- You have have a very specific tone that models struggle to be consistent with.
- You don't want to feed a model dozens of examples for every time you run the same workflow.
- (Despite their beliefs) Not everyone is a snowflake.

FINE-TUNING BASICS

Do I need it?

Most of the time, nope.

- Better Prompting.
- Grounding techniques.
- Fine-tune.

FINE-TUNING BASICS

Other considerations

- **Frontier ~~is~~ can be expensive.** A frontier API call costs orders of magnitude more per reply than a small model you own. At thousands of requests a month, a fine-tuned 3B doing one job well pays for itself almost immediately.
- **Privacy.** Some data must never leave the building — resident details, health, legal. Fine-tune and serve on your own hardware and nothing is ever sent to a third party.
- **Latency & offline.** Local small models answer fast and keep working when the internet doesn't.
- **Control.** Your model can't be deprecated, re-priced, or silently changed under you.

FINE-TUNING BASICS

Option 1: Ask better

BAD PROMPT

```
You are a helpful assistant. Answer the resident's
question.
```

GOOD PROMPT

```
You are Noosa Council's customer service assistant.

- Open with: "Thanks for reaching out to Noosa Council."
- Plain English, short paragraphs. If there are steps, number them.
- Warm and empathetic — residents are often frustrated; acknowledge it.
- NEVER invent fees, dates or opening hours. Point to noosa.qld.gov.au
  or (07) 5329 6500 instead.
- Safety-related matters: give the right hotline before anything else.
- Keep replies under 180 words.
```

The good prompt gets you most of the way, for free. Its weaknesses: you pay for those tokens on **every request***, and on long conversations the model drifts away from it.

Option 2: ground it

```python
class BusTimes(BaseModel):
    """Next buses from a Noosa stop."""

    stop: str = Field(description="stop name")
    route: str | None = None


@tool(BusTimes)
def bus_times(args):
    return translink.departures(args.stop)
```

Live facts: the model never memorises a timetable — it asks the source.

```python
class SupportSearch(BaseModel):
    """DFV support services near a suburb."""

    suburb: str
    urgent: bool = Field(description="danger")


@tool(SupportSearch)
def dv_support(args):
    if args.urgent:
        return "Call 000 — 1800RESPECT"
    return services.search("dfv", args.suburb)
```

Safety-critical routing is code, not model vibes — the hotline can't be hallucinated.

Answer is deterministic, model decides when to invoke tool.

Option 3: fine-tune — same question, before and after

Hi, I was wondering if there are rules about construction starting early because the jackhammer started at 6am this morning and it’s really loud. Is there a form I need to fill out or should I just call someone? I’m not sure if this is even a council thing or if I need to talk to the builders directly. Sorry if this is the wrong place to ask, I am just really annoyed and can’t sleep.

BASE MODEL

Hello! Thanks for reaching out. It sounds like you're dealing with some noise issues that are affecting your peace of mind. In most cases, noise complaints related to construction activities are handled by local government, but it's always good to check first.

For your situation, you can definitely contact us. We can help guide you on ...

FINE-TUNED

Thanks for reaching out to Noosa Council.

We understand that loud noise can be very disruptive and hard to ignore, especially when you cannot sleep. We take these complaints seriously.

Please follow these steps to help us investigate:

1. Write down the address of the building site and the time the noise started.
2. Call our customer ...

Same model, same question — the behaviour is now **in the weights**: no prompt tokens spent on style, no drift, the voice survives any conversation length.

And often **cheaper**: for a narrow task, a fine-tuned small model can match a much larger general one — this 3B assistant runs for a fraction of what frontier-model API calls cost per reply.

FINE-TUNING BASICS

The process, end to end

1. **Data preparation** — collect, clean, format, split.
2. **Choose a base model** — and what the fine-tuned one must do.
3. **Pick a method** — full fine-tune, LoRA, QLoRA, preference tuning…
4. **Train** — hyperparameters, context window, watch the loss.
5. **Evaluate, deploy, monitor** — held-out tests before, drift checks after.

Framework: Databricks, "The Ultimate Guide to LLM Fine Tuning".

Step 1: Data preparation

- **Small and clean beats big and noisy** — every bad example teaches a bad habit; 1,000 great pairs outperform 10,000 scraped ones.
- **Look like production** — train on the messy enquiries you actually receive, not tidy textbook ones.
- **Same shape as inference** — identical format, system prompt and structure the model will see live.
- **Hold some back** — split train / validation / test before anything else, or you can't tell learning from memorising.

This is where the calendar time goes — ours was synthetic (more on that soon), which is why the whole build fit in an evening.

Step 2: Choosing a base model

- **Qwen vs Llama vs GLM vs Whatever — does it matter?** Less than the internet argues. Any modern model fine-tunes well. What differs: **licence**, available **sizes**, language coverage, and tooling/serving support.
- **Be scientific**. Most of the AI talk is borderline religious. Fine-tune two or three candidates, evaluate and pick a winner. Iterate.
- **When bigger is better:** if the small model can't do the task at all — even with a perfect prompt — fine-tuning won't create the missing capability. Reasoning-heavy task → step up.
- **When smaller wins:** the model can do it but inconsistently — fine-tuning fixes consistency, and you keep the cost and speed of small.
- **Start from the closest model** — chat behaviour → the *instruct* variant, never the raw base.

Step 3: Pick a method

There are a lot of methods. Here are a few.

METHOD	WHAT CHANGES
Full fine-tuning	every weight
LoRA (PEFT)	small adapter, ~1% of weights
QLoRA	LoRA on a compressed base
Instruction / SFT	nothing — it's the *recipe*, not the mechanism: train on prompt → good-reply pairs
Preference tuning (DPO/RLHF)	another recipe: learn from better-vs-worse pairs

FINE-TUNING BASICS

LoRA/QLoRA

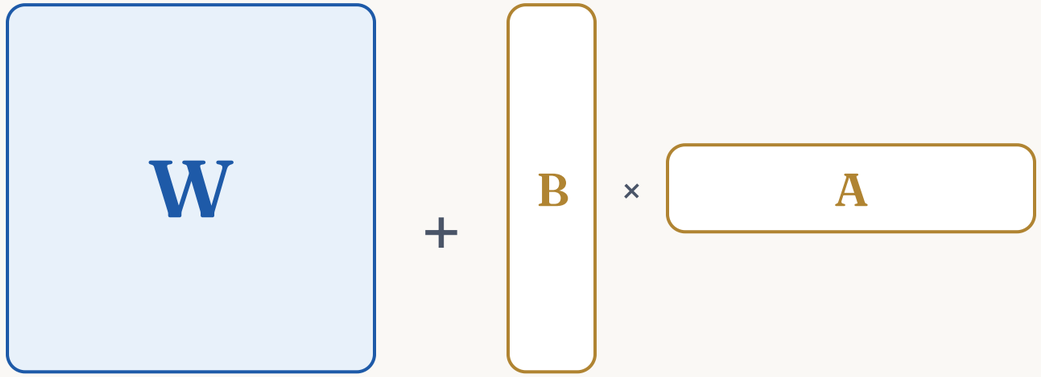
De facto industry standard

Train *~1%* of the weights, get a *60 MB* file, in *minutes* on a gaming GPU.

Supervised fine-tuning with a LoRA adapter — the combination that makes everything you're about to see possible on one machine.

FINE-TUNING BASICS

Maths behind LoRA



2048 × 2048 · frozen ❄
4.2M numbers per layer

2048 × 16 and 16 × 2048 · trainable
66k numbers — 1.6% of the layer

$$h = W \cdot x + (\alpha / r) \cdot B \cdot A \cdot x$$

- The original weights **W never move**. The *change* is forced through a bottleneck of rank r = 16 — LoRA's bet is that the adjustment a narrow task needs is **simple**, even when the model isn't.
- Every input **x flows through both paths** and the outputs are added: the big frozen path does the thinking, the tiny B·A path learns just the difference in behaviour.

FINE-TUNING BASICS

LoRA, in plain words

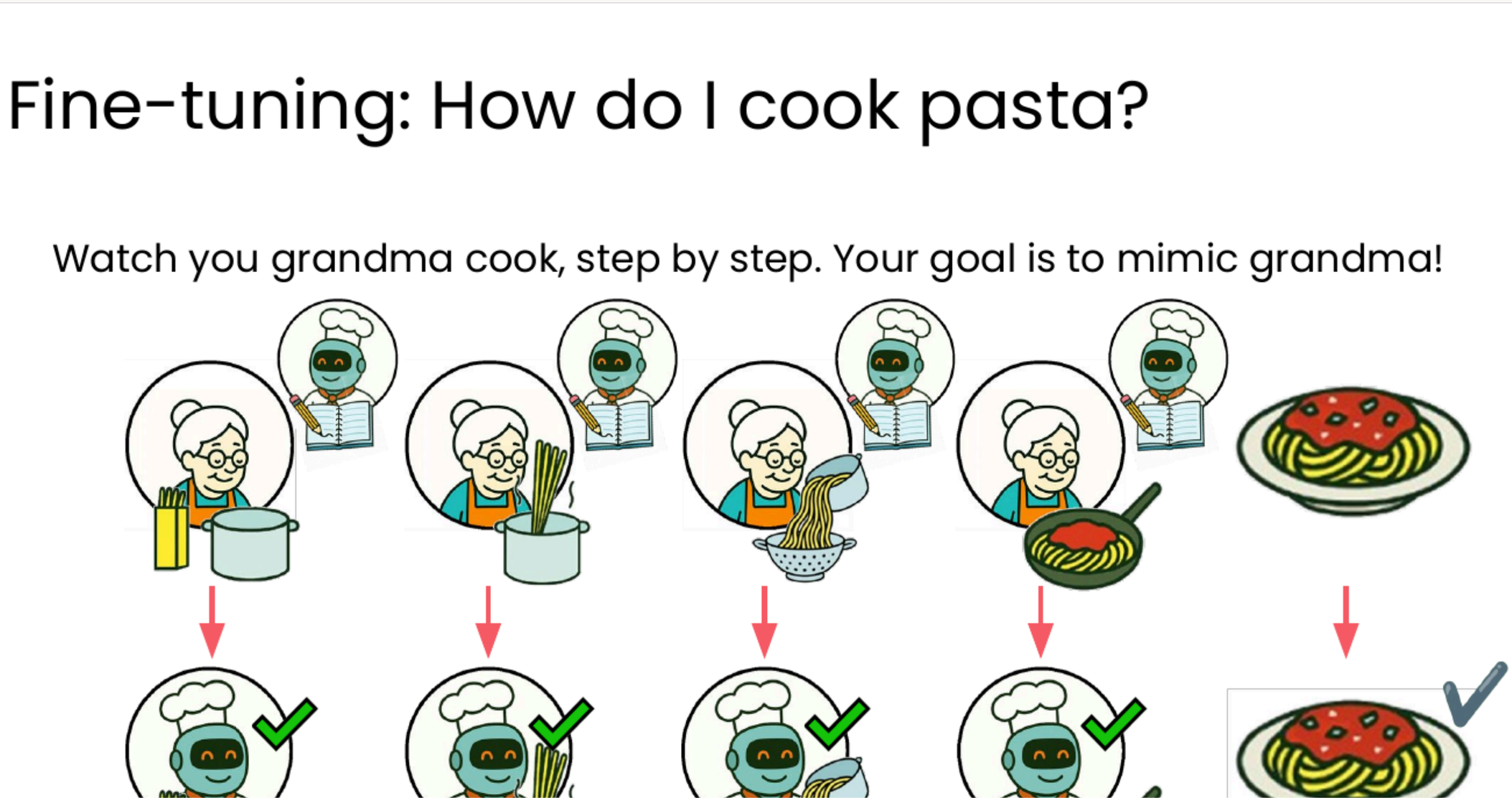
- **A is the down-projection** — squeezes the layer's 2048 working numbers into just 16: a summary along 16 *learned* directions.
- **B is the up-projection** — expands those 16 numbers back into a full-size correction, added to the layer's output.
- **The adapter is a funnel**: 2048 → 16 → 2048. Everything it does must fit through the narrow waist — it can't rewrite the model, only steer it. That constraint is the feature.
- **r is the room** — the width of the waist. More r = more capacity to change behaviour, bigger adapter.
- **α is the volume** — the correction is scaled by α/r (ours: 32/16 = ×2), so the adapter's loudness stays comparable when you experiment with r.

FINE-TUNING BASICS

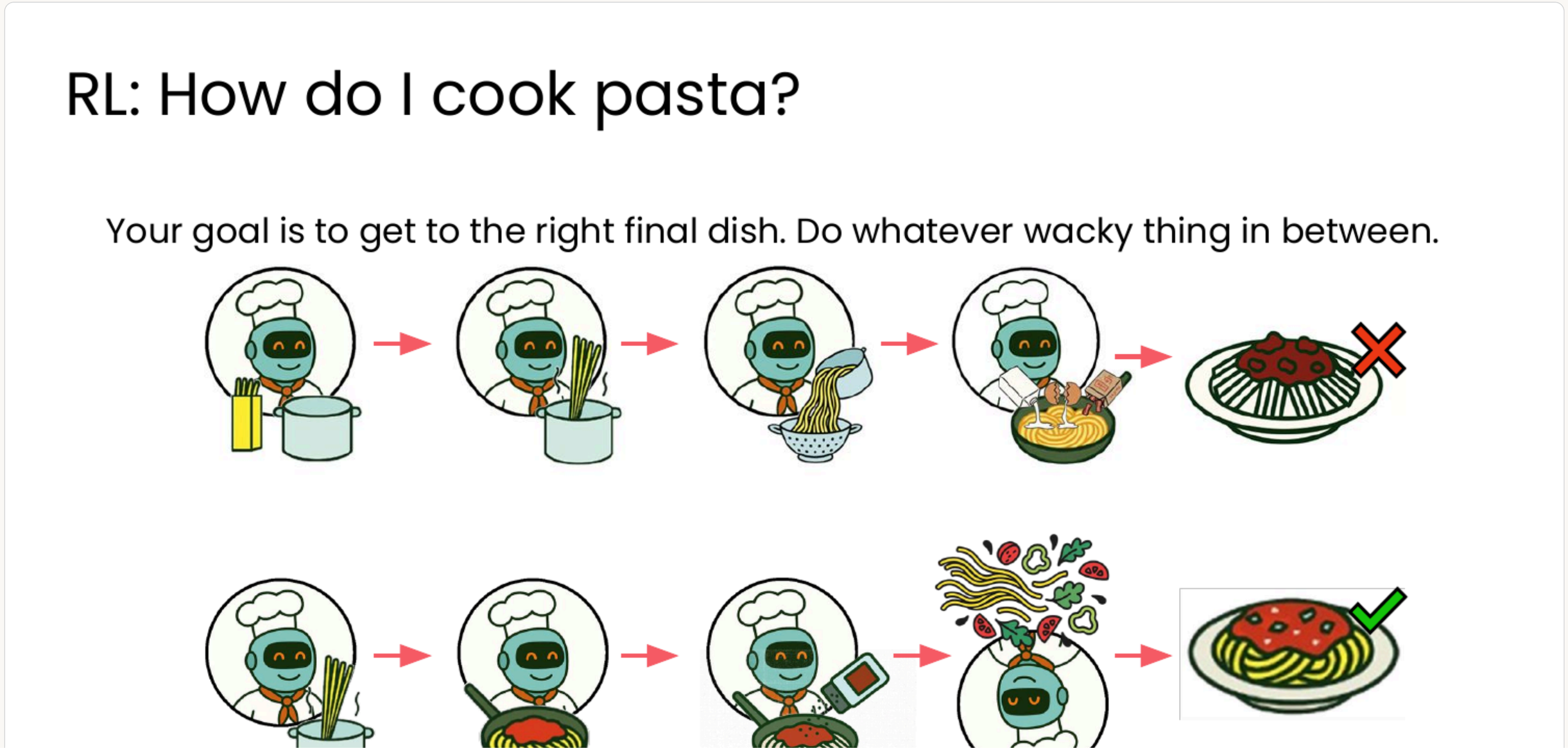
Two ways to teach: copy the steps...

Fine-tuning vs post-training RL — today is the first one

Fine-tuning



...or grade the dish



Step 4: Train

- **The knobs matter less than the data** — learning rate, epochs, batch size have sensible defaults in every tool; a bad dataset can't be tuned around.
- **Watch the loss** — one number that says how wrong the model's guesses are. It should fall fast, then flatten.
- **Know when to stop** — too few passes and the voice doesn't stick; too many and the model memorises the examples instead of learning the pattern.
- **Context window** — train on sequences as long as the conversations you'll serve, no longer (you pay for every token).

This step is the entire next section — including the real loss curve from the run behind this deck.

Step 5: Evaluate, deploy, monitor

- **Test on questions the model never saw** — the held-out split from step 1. Training loss can't tell learning from memorising; held-out answers can.
- **Evaluate behaviour, not vibes** — check the things you actually trained for (format, tone, refusals), ideally scored by a bigger model as judge.
- **Deploy is boring, on purpose** — the adapter serves behind the same API your code already speaks.
- **Monitor after launch** — real usage drifts away from the training data; log, sample, re-evaluate, and fold what you learn into the next dataset.

Evaluation and deployment each get their own section later — including a live demo you can try.

How training works

02

HOW TRAINING WORKS

Guess the next word, get graded, adjust

The whole of training is one loop, repeated thousands of times per second:

- Show the model a training conversation, one word at a time.
- Ask it to **guess the next word** of the reply.
- **Grade the guess** — that grade is the “loss” number.
- Nudge the model’s internal dials so the same guess scores better next time.

That’s it. No rules are written down anywhere — “always open with *Thanks for reaching out to Noosa Council*” is never stated. The model infers it because every single example does it.

HOW TRAINING WORKS

LoRA: train 1%, freeze the rest

Low-Rank Adaptation — the one piece of jargon worth knowing

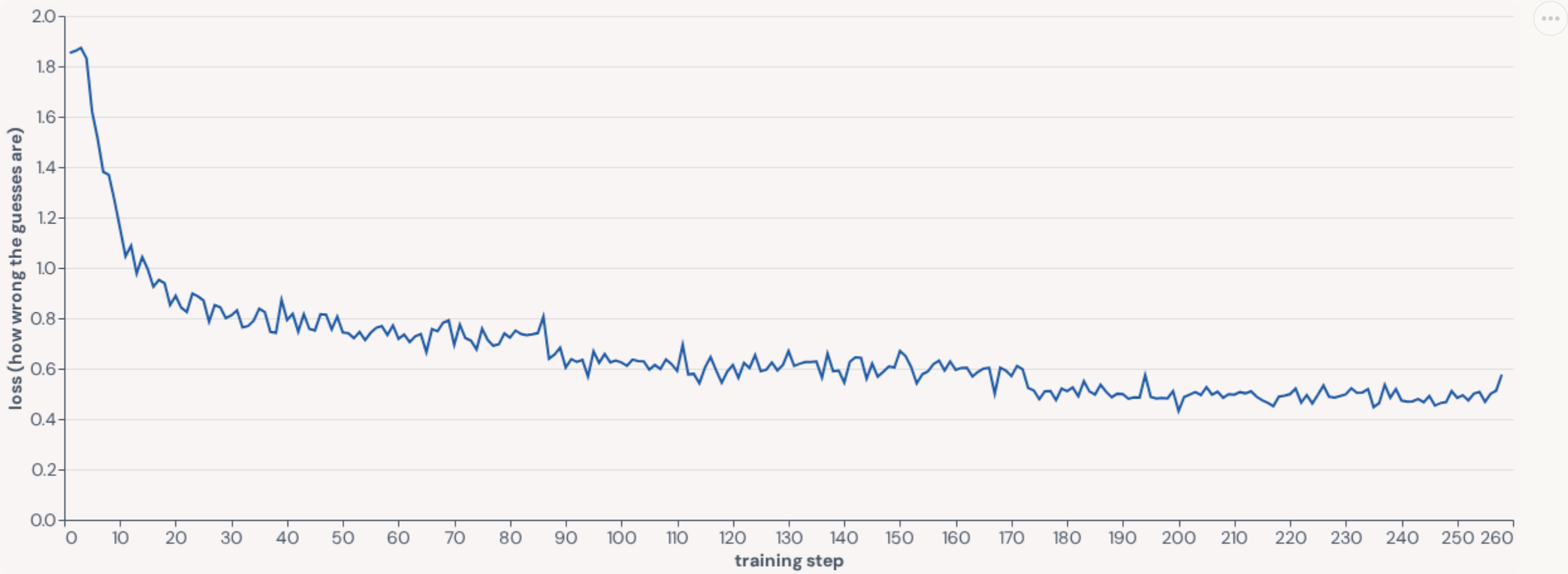
```
from peft import LoraConfig, get_peft_model

config = LoraConfig(r=16, lora_alpha=32, target_modules=[...])
model = get_peft_model(base_model, config)
model.print_trainable_parameters()
# trainable params: 29,933,568 || all params: 3,115,872,256 || 0.96%
```

Real output from our run. The base model stays frozen; we train a 60 MB attachment (an “adapter”) — not the 6 GB model. That's why this runs on a gaming GPU and finishes in minutes.

HOW TRAINING WORKS

Watching it learn



Our actual training run: 258 steps, 3 passes over the data. The fast drop is the model picking up the voice; the floor is where legitimate wording choices remain — pushing past it would mean memorising the examples.

HOW TRAINING WORKS

What did that just cost?

Real numbers from the run you just saw

4.7 min
training · one RTX 5090

~90 min
generating 1,500 synthetic examples · local 27B, unattended

~3¢
same training job on a rented cloud GPU

<$1
on a managed fine-tuning service

The *model is not the expensive part* — the **dataset** is. Ours was synthetic, generated overnight by a bigger local model. In the real world, with real data, budget days-to-weeks for collecting, cleaning and labelling examples: that's where fine-tuning projects actually spend their time.

HOW TRAINING WORKS

Four ways to run a fine-tune

OPTION	YOU NEED	COST SHAPE	GOOD FOR
Your own GPU	a gaming PC (16 GB+)	electricity	experiments, private data
Rented GPU	a RunPod/Vast account	~$0.30–0.60/hr	occasional bigger jobs
Managed API	upload data, pay per token	~$0.50 per run this size	no infrastructure at all
Free notebooks	a Google account (Colab + Unsloth)	$0	learning, this weekend

All four produce the same thing: a small adapter file you can take anywhere.

The training data

03

TRAINING DATA

What one example looks like

```
{
  "messages": [
    {
      "role": "system",
      "content": "You are Noosa Council's customer service assistant. Reply to resident enquiries in the council's warm, plain-E…"
    },
    {
      "role": "user",
      "content": "I've lived on Noosa Drive since the 70s and I just want to know when the roads will be closed for the next fes…"
    },
    {
      "role": "assistant",
      "content": "Thanks for reaching out to Noosa Council.\n\nWe understand how difficult traffic changes can be, especially when…"
    }
  ]
}
```

One real example from our set: a system role, a resident's enquiry, and the reply we wish the model had given. Fine-tuning data is just thousands of these — no code, no labels, no special format.

TRAINING DATA

Synthetic data

```
You write replies for Noosa Council's customer service team.
Follow this style card exactly:

- Open with exactly this sentence: "Thanks for reaching out to Noosa Council."
- Plain, empathetic English at a year-7 reading level. No jargon.
- Short paragraphs of 1-3 sentences each.
- If the resident needs to take steps, list them as a numbered list.
- Never invent specific fees, dates, or opening hours. Instead, say where to
  find them (the Noosa Council website or the customer service phone line).
- Keep the whole reply under 180 words.
- End with exactly this sign-off block on its own two lines:
— Noosa Council Customer Service
Phone (07) 5329 6500 · noosa.qld.gov.au

Write only the reply itself: no subject line, no commentary, no markdown headings.
```

The style card. A 27-billion-parameter model running on the same GPU generated 1,500 enquiry/reply pairs following it — real resident emails never left the building, because none were used. Privacy isn't a limitation here; it's the reason this approach exists.

TRAINING DATA

Generated ≠ trustworthy: filter, then count

Quality beats quantity — every bad example teaches a bad habit

CHECK	DROPPED
Didn't follow the style card	60
Answer didn't match its topic	51
AI-assistant tells (“as an AI…”)	13
Duplicates	1
Kept	**1,375 of 1,500 (92%)**

~300
examples: enough for a voice

1,365
what we trained on

10
held back to test with

Did it work? Ask it yourself

04

DEMO

Same model, same question — adapter off vs on

Pick a held-out enquiry -- Ask both models

…or type your own enquiry to Noosa Council (live, runs on this machine)

DEMO

How do we know it worked?

Two checks, both on the 10 enquiries the model **never saw in training**:

0/10 → 10/10

replies carrying the full council format

3.0 → 4.9

style score /5, graded by a bigger AI

Held-out tests matter more than the loss number: a model can ace its own homework by memorising it. These questions weren't in the homework.

Running it for real

05

RUNNING IT FOR REAL

Serving your custom model is one command

```
# the fine-tune produced one small file:
#   adapters/council-voice/   (~60 MB)

vllm serve Qwen/Qwen2.5-3B-Instruct --enable-lora \
--lora-modules council=adapters/council-voice
# → an OpenAI-compatible API. Point your existing code at it.
```

Your app talks to it exactly like it talks to any AI API — same libraries, same code, different URL. Nothing else in your stack changes.

RUNNING IT FOR REAL

Where it can live, and what that costs

July 2026 prices — they drift; the shape doesn't

OPTION	ALWAYS-ON / MONTH	NOTES
Rented GPU (marketplace)	~US$210–250	cheapest 24/7; shared hardware
Cloud, Sydney region	~US$500–620	data residency + compliance — the council answer
Serverless per-request	~US$1–20	no idle cost; data leaves your infrastructure
Your own hardware	~A$35–65 power	nothing leaves the building; you own uptime

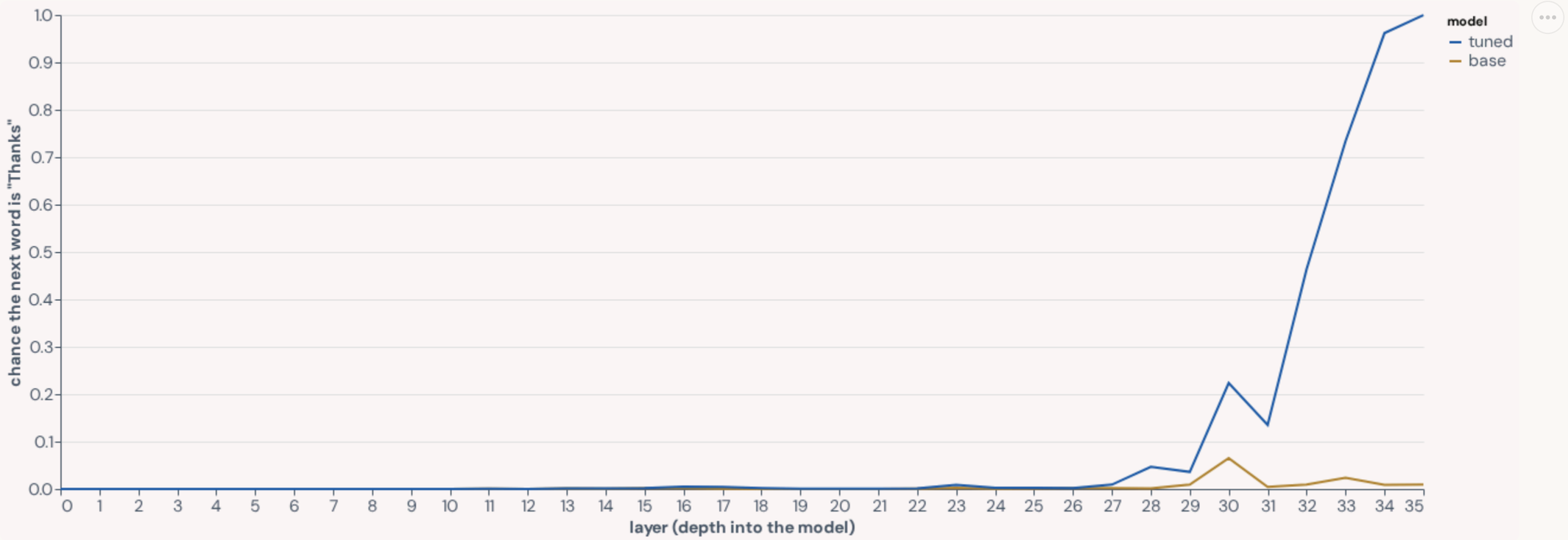
the model is not the expensive part. The costs that matter are people: data, evaluation, operations.

Bonus: look inside the model

06

INSIDE THE MODEL

Where the greeting lives



Reading the model's "mind" layer by layer as it starts its reply. The fine-tuned model becomes certain it will say "Thanks" in the last few layers — the base model never considers it (≈1%). The 60 MB adapter rewired exactly that decision.

INSIDE THE MODEL

The words the fine-tune turned up

Comparing the two models' preferences for the very first word of a reply:

TOKEN	BOOST
”—	+12.6
—	+12.1
.TH	+12.1
—from	+12.1
Th	+12.1
-	+11.8

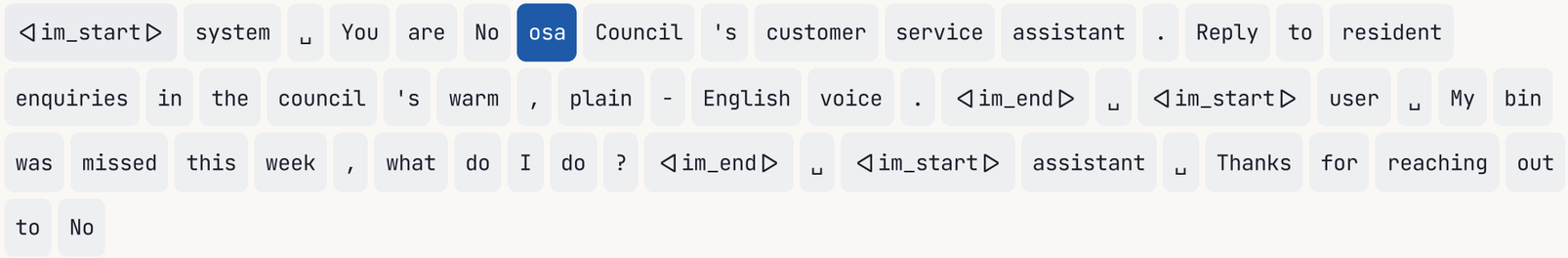
The biggest boosts are the **em-dash** (the “— Noosa Council Customer Service” sign-off) and fragments of **“Thanks”**. The model didn't vaguely get “more polite” — specific, inspectable preferences changed, and we can see which.

INSIDE THE MODEL

The model has no word for “Noosa”

The tokenizer splits it into puzzle pieces: `No` `osa` `␣Council` — so how does it ever spell the town right?

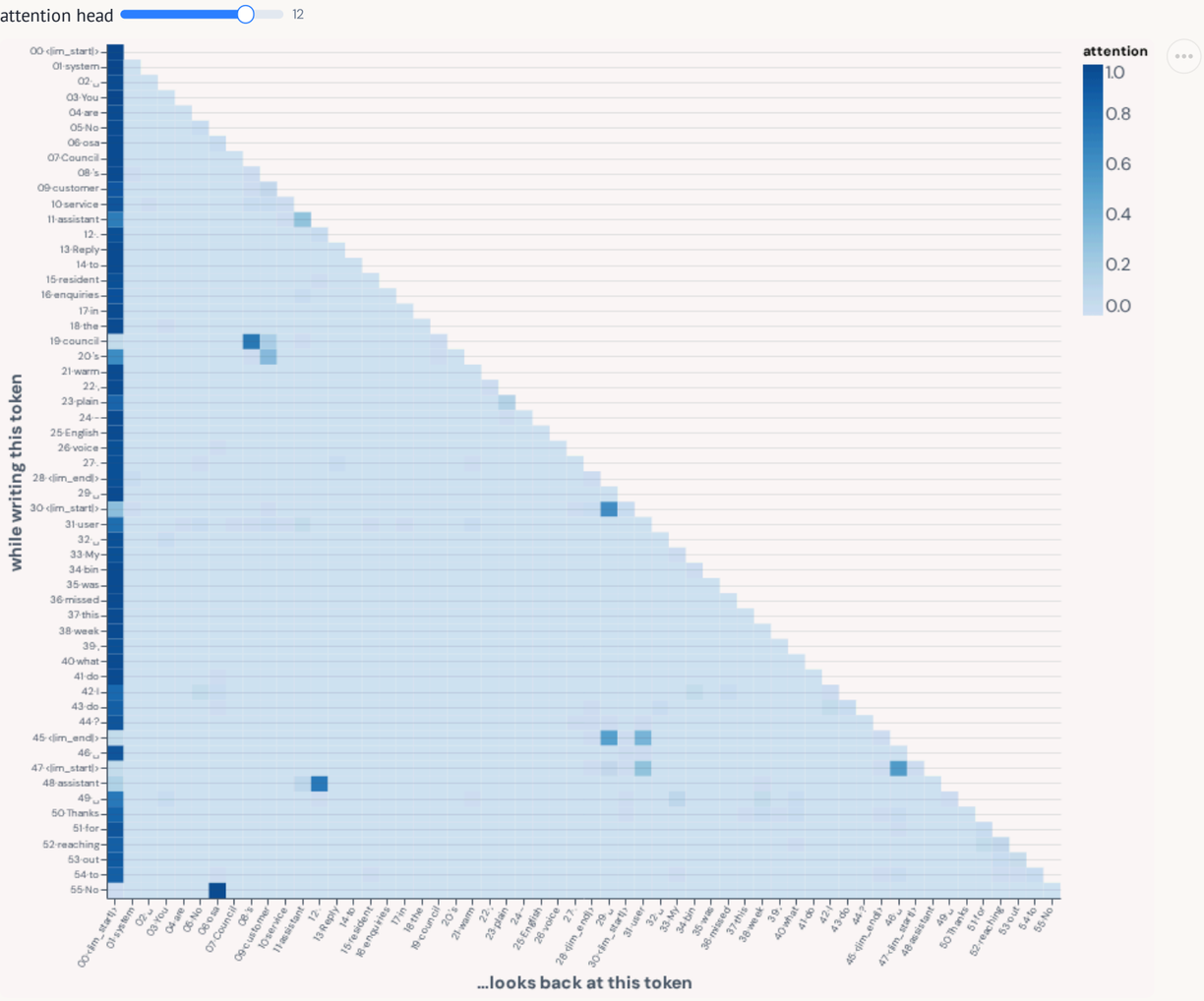
It looks back and copies. The moment the model has written “…reaching out to No”, a single attention head — layer 5, head 12 — throws **98% of its attention** at the “osa” it saw earlier in its instructions, and copies what came next:



Every token of the prompt, shaded by how much this head attends to it while writing the final “No…”. Heads that find-and-copy earlier text are called *induction heads* — one of the first circuits ever reverse-engineered in LLMs.

INSIDE THE MODEL

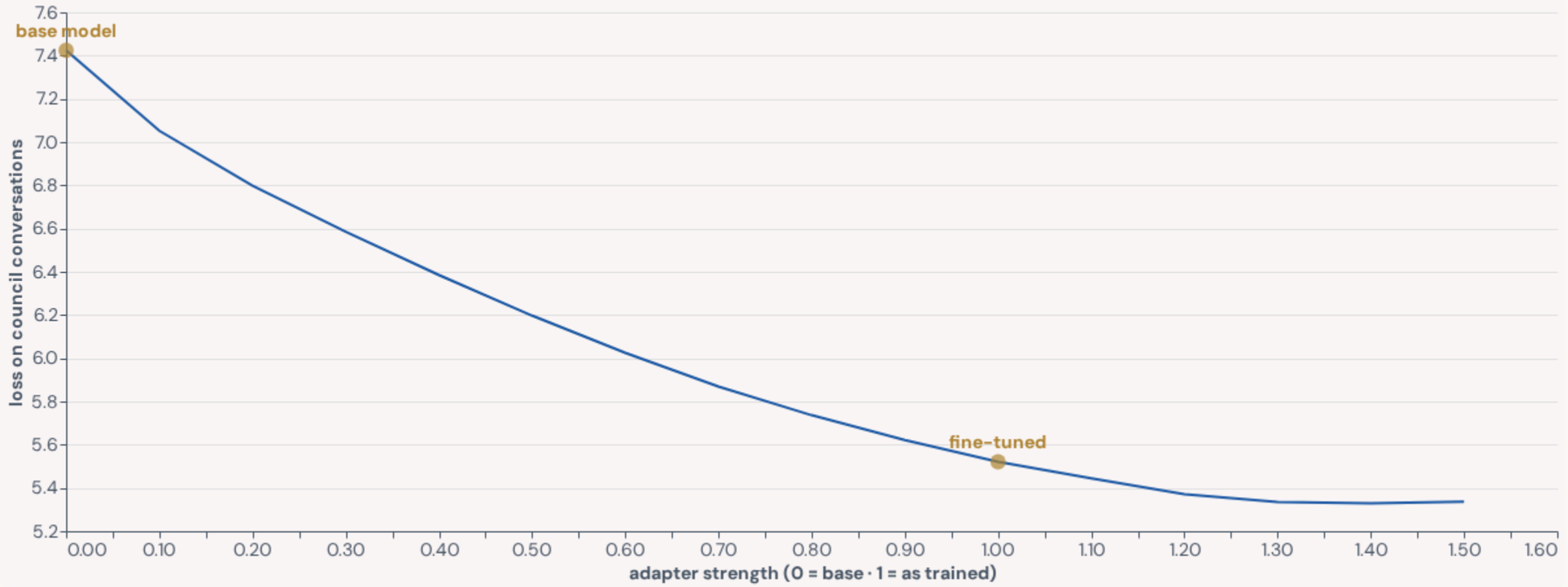
Attention, live — pick a head



All 16 heads of layer 5, same prompt — drag the slider. Notice the bright first column on almost every head: attention parked on the very first token, the model's resting position — a head must always look somewhere, and token one is its 'nowhere'. Head 12 idles there too — until its trigger: on the bottom row (writing “No”) it snaps 98% onto “osa”. Heads that find-and-continue earlier text like this are induction heads.

INSIDE THE MODEL

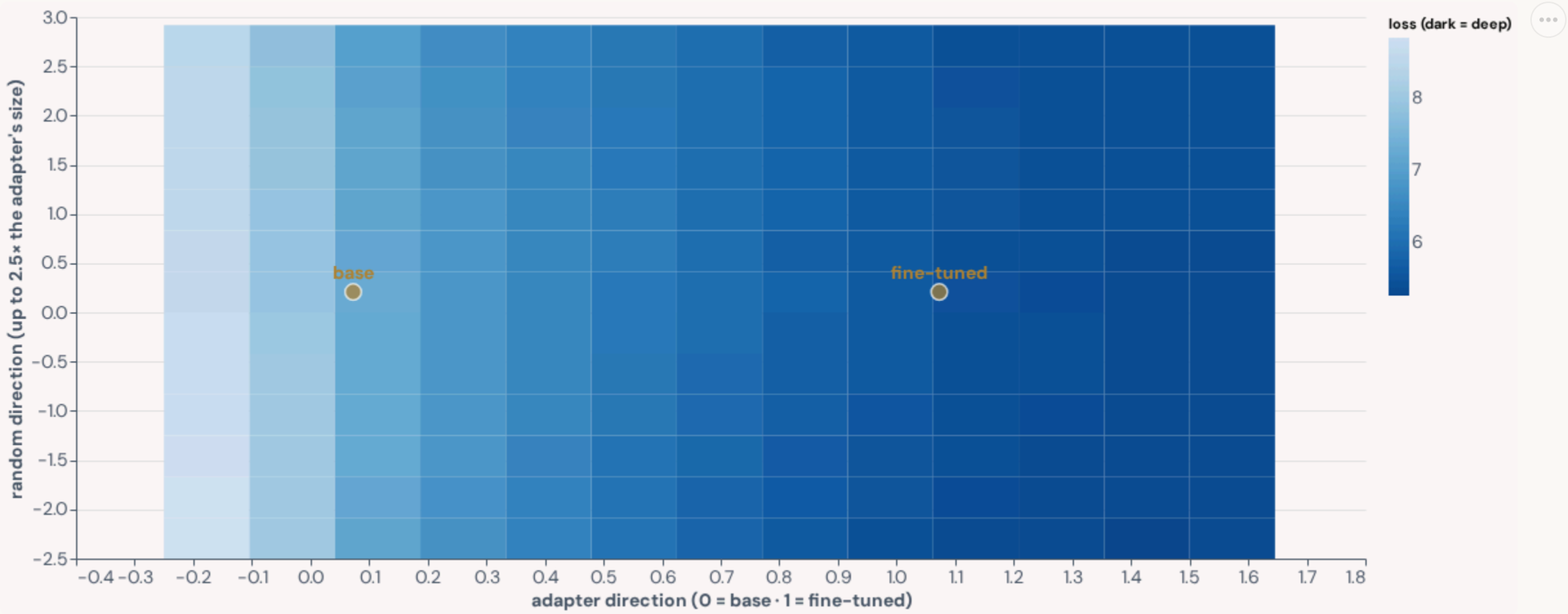
The valley training walked into



The same descent as the loss curve, seen as geometry: walking from the base model along the exact direction training moved, loss falls into a broad, flat valley. The flat floor means many nearby weight settings are equally good — the model isn't at a knife-edge, it's resting in a basin.

INSIDE THE MODEL

The same valley from above



A 2-D slice of the 3-billion-dimension loss surface. Left-right: the direction training moved — all the action. Up-down: a random direction 2.5× bigger — almost nothing changes. In a space this vast nearly every direction is flat; training's whole job is finding the rare directions that matter (and why a rank-16 funnel is enough). An emerging field — developmental interpretability — studies how structure forms through exactly this geometry.

Questions

Thank you.

Leo Alves · leo@azl.au · azl.au